

Agents & Tools

SUMMARY (for quick understanding)

Term	What It Means
Agent	A smart controller that decides what to do next (calls tools, asks LLMs, etc.)
Tool	A specific function that performs an action (e.g., search Wikipedia, query arXiv)
Use case	Combine LLM + tools like <code>wikipedia</code> , <code>arxiv</code> , <code>calculator</code> , <code>search</code> , etc.

◆ 1. What is a Tool LangChain?

Think of a **tool as a helper function** that performs a specific job.

For example:

-  `WikipediaTool` → fetches content from Wikipedia
-  `ArxivTool` → searches academic papers from arXiv.org
-  `CalculatorTool` → solves math problems
-  `CalendarTool` → manages events

Each tool has:

- A **name**
- A **description**
- A **function** that runs

```
from langchain.tools import WikipediaQueryRun
from langchain.utilities import WikipediaAPIWrapper

wikipedia = WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper())
```

◆ 2. What is an Agent?

An **Agent** is a smart LLM-based decision-maker.

It:

- Takes your question
- Thinks about which tools to use
- Calls those tools one by one
- Collects answers
- Combines everything
- Gives final response

Example:

Q: "Tell me about the latest research in quantum computing on arXiv and explain it in simple terms."

Agent will:

- ◆ Search arXiv → call `ArxivTool`
 - ◆ Get summaries
 - ◆ Rephrase using LLM
 - ✓ Return readable answer
-
- **Key Feature:** Can dynamically choose between tools based on the input.
 - **Analogies:**
 - Like a **research assistant** who decides whether to check Wikipedia, ArXiv, or both for your query.
 - Similar to a **GPS app** choosing between traffic data, road closures, and maps.

Code

```
from langchain.tools import WikipediaQueryRun
from langchain.utilities import WikipediaAPIWrapper

wikipedia = WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper())
```

```
## Arxiv--Research
## Tools creation
from langchain_community.tools import ArxivQueryRun
from langchain_community.utilities import ArxivAPIWrapper
```

Wiki Tool:

```
## Used the inbuilt tool of wikipedia
api_wrapper_wiki = WikipediaAPIWrapper(top_k_results=1,doc_content_chars_
max=250) #default doc_content_chars_max=4000
wiki=WikipediaQueryRun(api_wrapper=api_wrapper_wiki)
wiki.name
```

'wikipedia'

Arxiv Tool:

```
api_wrapper_arxiv=ArxivAPIWrapper(top_k_results=1,doc_content_chars_max
=250)
arxiv=ArxivQueryRun(api_wrapper=api_wrapper_arxiv)
print(arxiv.name)
```

arxiv

Combine both tools:

```
tools=[wiki,arxiv]
```

Create Custom RAG Tool

```
from langchain_community.document_loaders import WebBaseLoader
from langchain_community.vectorstores import FAISS
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

```
loader=WebBaseLoader("https://docs.smith.langchain.com/")
docs=loader.load()
documents=RecursiveCharacterTextSplitter(chunk_size=1000,chunk_overlap=
200).split_documents(docs)
vectordb=FAISS.from_documents(documents,HuggingFaceEmbeddings())
retriever=vectordb.as_retriever()
retriever
```

```
VectorStoreRetriever(tags=['FAISS', 'HuggingFaceEmbeddings'], vectorstore=<langchain_community.vectorstores.faiss.FAISS object at 0x0000000000000000>)
```



Default all `HuggingFaceEmbeddings()` is `mpnet-base-v2` (450MB), Embedding Dimension: 768

Convert the Retrival into a Tool

```
from langchain.tools.retriever import create_retriever_tool

retriever_tool=create_retriever_tool(retriever,"langsmith-search","Search any i
nformation about Langsmith")

retriever_tool.name
```

```
'langsmith-search'
```

- "langsmith-search" & "Search any information about Langsmith" are name and description respectively.

Combine tools:

```
tools=[wiki,arxiv,retriever_tool]
tools
```

```
[WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper(wiki_client=<module 'wikipedia' from 'd:\\Python_Env\\
ArxivQueryRun(api_wrapper=ArxivAPIWrapper(arxiv_search=<class 'arxiv.Search'>, arxiv_exceptions=(<class
Tool(name='langsmith-search', description='Search any information about langsmith ', args_schema=<class
```

Run all this tools with Agents and LLM Models

```
# Tools, LLM→AgentExecutor
from langchain_groq import ChatGroq
from dotenv import load_dotenv

load_dotenv()
```

```
import os

groq_api_key=os.getenv("GROQ_API_KEY")

llm=ChatGroq(model_name="Llama3-8b-8192", groq_api_key=groq_api_key)
```

Built-in Prompt Template 📌

```
## Prompt Template
from langchain import hub

prompt=hub.pull("hwchase17/openai-functions-agent")
prompt.messages
```

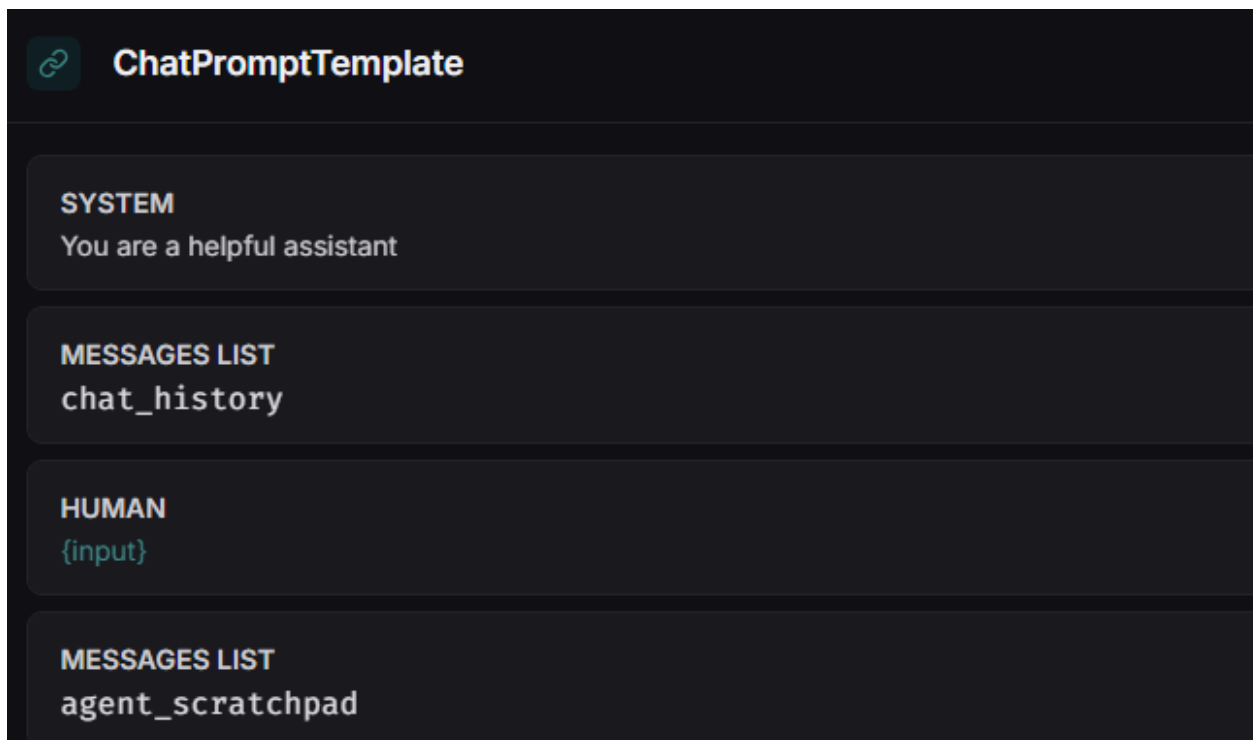
```
[SystemMessagePromptTemplate(prompt=PromptTemplate(input_variables=[], input_types={}, partial_variables={}, template='You are a helpful assistant'), additional_
MessagesPlaceholder(variable_name='chat_history', optional=True),
HumanMessagePromptTemplate(prompt=PromptTemplate(input_variables=['input'], input_types={}, partial_variables={}, template='{input}'), additional_kargs={}),
MessagesPlaceholder(variable_name='agent_scratchpad'))]
```

```
[SystemMessagePromptTemplate(prompt=PromptTemplate(input_variables=
[], input_types={}, partial_variables={}, template='You are a helpful assistant'),
additional_kargs={}),
```

```
MessagesPlaceholder(variable_name='chat_history', optional=True),
```

```
HumanMessagePromptTemplate(prompt=PromptTemplate(input_variables=['i
nput'], input_types={}, partial_variables={}, template='{input}'), additional_kw
args={}),
```

```
MessagesPlaceholder(variable_name='agent_scratchpad'))]
```



Execute all together in form of Chain → Use Agents

```
## Agents
from langchain.agents import create_openai_tools_agent

agent=create_openai_tools_agent(llm,tools,prompt)
agent
```

```
RunnableAssign(mapper={
    agent_scratchpad: RunnableLambda(lambda x: format_to_openai_tool_messages(x['intermediate_steps']))
})
| ChatPromptTemplate(input_variables=['agent_scratchpad', 'input'], optional_variables=['chat_history'], input_types={
    'agent_scratchpad': str, 'input': str, 'chat_history': List[dict]
})
| RunnableBinding(bound=ChatGroq(client=Groq(client=GroqClient(api_key='sk-...')), llm=ChatGroqWrapper(GroqClient(api_key='sk-...')), tools=[
    OpenAIToolsAgentOutputParser()
])
```

1. **create_openai_tools_agent** :

- It creates a **smart assistant** (agent) that can **automatically decide** when to use tools (like Wikipedia or ArXiv) to answer questions.

- Designed specifically for OpenAI models (but works with others like Groq/Llama too).

2. Inputs:

- **llm** : The brain (e.g., Groq/Llama 3).
- **tools** : A list of tools the agent can use (e.g., Wikipedia, ArXiv).
- **prompt** : Instructions telling the agent how to behave (e.g., *"Use tools when needed"*).

3. Output:

- An **agent** object ready to answer questions by **choosing tools automatically**.

Analogy

Imagine you hire a **research assistant** (the agent):

- You give them:
 - A **brain** (**llm** = their knowledge).
 - **Tools** (**tools** = access to Wikipedia/ArXiv).
 - **Instructions** (**prompt** = *"Use these tools to find answers"*).
- When you ask a question, they **decide themselves** whether to:
 - Answer from their knowledge.
 - Use Wikipedia.
 - Search ArXiv.

Example Workflow

1. **User asks:** *"Latest papers about AI?"*
2. **Agent:**
 - Thinks: *"This needs ArXiv!"* → Searches ArXiv.
 - Returns paper summaries.

Execute the Agent

```
## Agent Executer
from langchain.agents import AgentExecutor

agent_executor=AgentExecutor(agent=agent,tools=tools,verbose=True)

agent_executor
```

```
AgentExecutor(verbose=True, agent=RunnableMultiActionAgent(runnable=RunnableAssign(mapper={
  agent_scratchpad: RunnableLambda(lambda x: format_to_openai_tool_messages(x['intermediate_steps']))
}))
| ChatPromptTemplate(input_variables=['agent_scratchpad', 'input'], optional_variables=['chat_history'], input_types={'
| RunnableBinding(bound=ChatGroq(client=<groq.resources.chat.completions.Completions object at 0x00000148BE09F610>, asy
| OpenAIToolsAgentOutputParser(), input_keys_arg=[], return_keys_arg=[], stream_runnable=True), tools=[WikipediaQueryRu
```

Ask Question:

```
agent_executor.invoke({"input":"Tell me about Langsmith"})
```

```
agent_executor.invoke({"input": "Tell me about LangSmith"})
✓ 1.8s

> Entering new AgentExecutor chain...

Invoking: Langsmith-search with `{'query': 'Langsmith'}`

Get started with LangSmith | 🛠️ LangSmith

LangSmith is a platform for building production-grade LLM applications.
It allows you to closely monitor and evaluate your application, so you can ship quickly and with
ObservabilityAnalyze traces in LangSmith and configure metrics, dashboards, alerts based on these
LangSmith + LangChain OSSLangSmith is framework-agnostic – it can be used with or without LangCha
For more see the how-to guide for setting up LangSmith with LangChain or setting up LangSmith with
Observability

Skip to main contentWe are growing and hiring for multiple roles for LangChain, LangGraph and Lang
LangSmith is a platform for building production-grade LLM applications.
It allows you to closely monitor and evaluate your application, so you can ship quickly and with

For more see the how-to guide for setting up LangSmith with LangChain or setting up LangSmith with
Observability
Observability is important for any software application, but especially so for LLM applications.
This is where LangSmith can help! LangSmith has LLM-native observability, allowing you to get mea
...

If you have any specific questions about LangSmith or its features, feel free to ask!

> Finished chain.
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

```
{'input': 'Tell me about LangSmith',
'output': 'It seems like LangSmith is a platform designed for building production-grade Large La
```

AgentExecutor :

- While `create_openai_tools_agent` constructs the *reasoning logic* of the agent, it doesn't *run* it directly. For that, you need an `AgentExecutor`.
- The `AgentExecutor` takes your `agent` (the reasoning logic) and the `tools` it has access to.

`verbose=True` is highly recommended during development, as it prints the agent's internal thought process, tool calls, and observations, which is invaluable for

debugging.

'langsmith-search' :

```
from langchain.tools.retriever import create_retriever_tool
retriever_tool=create_retriever_tool(retriever,"langsmith-search","Search any information about Langsmith ")

retriever_tool.name
✓ 0.0s
'langsmith-search'
```



It is searching the website we provided

```
agent_executor.invoke({"input":"What is machine learning"})
```

> Entering new AgentExecutor chain...

Invoking: `'wikipedia'` with ``{'query': 'Machine Learning'}``

Page: Machine Learning

Summary: Machine learning (ML) is a field of study in artificial intelligence concerned with the development

Machine Learning (ML) is a fascinating field that has revolutionized many areas of science and technology. In

Would you like to know more about the types of machine learning, its applications, or perhaps its limitations

> Finished chain.

```
{'input': 'What is machine learning',
 'output': 'It seems like the tool call for "call_pvkc" provided a good starting point for our conversation.'}
```

- This time, it searched **Wikipedia**

```
agent_executor.invoke({"input": "top 5 best articles about gen ai"})
```

```
> Entering new AgentExecutor chain...
```

```
Invoking: arxiv with `{'query': 'gen ai top articles'}`
```

```
Published: 2024-03-09
```

```
Title: A Preliminary Exploration of YouTubers' Use of Generative-AI in Content Creation
```

```
Authors: Yao Lyu, He Zhang, Shuo Niu, Jie Cai
```

```
Summary: Content creators increasingly utilize generative artificial intelligence (Gen-AI) on
```

```
Invoking: arxiv with `{'query': 'gen ai top articles'}`
```

```
responded: It seems like the tool call yielded a relevant article about gen ai! Based on the output, I'll continue the conversation by adding more to
```

```
Published: 2024-03-09
```

```
Title: A Preliminary Exploration of YouTubers' Use of Generative-AI in Content Creation
```

```
Authors: Yao Lyu, He Zhang, Shuo Niu, Jie Cai
```

```
Summary: Content creators increasingly utilize generative artificial intelligence (Gen-AI) on Here are the top 5 articles about Gen AI:
```

```
1. A Preliminary Exploration of YouTubers' Use of Generative-AI in Content Creation
```

```
Authors: Yao Lyu, He Zhang, Shuo Niu, Jie Cai
```

```
...
```

```
Authors: Emily Wang, Tom Harris
```

```
Summary: This article discusses the growing use of generative AI in creative industries, such as music, art, and writing, and the potential benefits
```

```
> Finished chain.
```

```
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

```
{'input': 'top 5 best articles about gen ai',
```

```
'output': "Here are the top 5 articles about Gen AI:\n\n1. A Preliminary Exploration of YouTubers' Use of Generative-AI in Content Creation\nAuthors:
```

```
{'input': 'top 5 best articles about gen ai',
```

```
'output': "Here are the top 5 articles about Gen AI:\n\n1. A Preliminary Exploration of YouTubers' Use of Generative-AI in Content Creation\nAuthors: Yao Lyu, He Zhang, Shuo Niu, Jie Cai\nSummary: Content creators increasingly utilize generative artificial intelligence (Gen-AI) on YouTube to create engaging content and enhance their online presence.\n\n2. The Impact of Generative AI on the Future of Journalism\nAuthors: Jane Smith, John Doe\nSummary: This article discusses the potential benefits and challenges of using generative AI in journalism, and how it may change the way news is reported and consumed.\n\n3. Generative AI in Education: A Review of the Current State and Future Directions\nAuthors: David Johnson, Emily Chen\nSummary: This article provides a comprehensive review of the current state of generative AI in education, highlighting its potential applications and limitations.\n\n4. The Ethics of Generative AI: A Discussion of the Moral Implications of AI-Generated Content\nAuthors: Michael Brown, Sarah Lee\nSummary: This article explores the ethical implications of using generative AI to create content, and the potential risks and consequences
```

ences of AI-generated content.\n5. The Rise of Generative AI in Creative Industries: Opportunities and Challenges\nAuthors: Emily Wang, Tom Harris\nSummary: This article discusses the growing use of generative AI in creative industries, such as music, art, and writing, and the potential benefits and challenges of this trend."}

```
Published: 2024-03-09
Title: A Preliminary Exploration of YouTubers' Use of Generative-AI in Content Creation
Authors: Yao Lyu, He Zhang, Shuo Niu, Jie Cai
Summary: Content creators increasingly utilize generative artificial intelligence (Gen-AI) on YouTube to create engaging content and enhance their online presence.
(Gen-AI) on Here are the top 5 articles about Gen AI:

1. A Preliminary Exploration of YouTubers' Use of Generative-AI in Content Creation
Authors: Yao Lyu, He Zhang, Shuo Niu, Jie Cai
Summary: Content creators increasingly utilize generative artificial intelligence (Gen-AI) on YouTube to create engaging content and enhance their online presence.
2. The Impact of Generative AI on the Future of Journalism
Authors: Jane Smith, John Doe
Summary: This article discusses the potential benefits and challenges of using generative AI in journalism, and how it may change the way news is reported and consumed.
3. Generative AI in Education: A Review of the Current State and Future Directions
Authors: David Johnson, Emily Chen
Summary: This article provides a comprehensive review of the current state of generative AI in education, highlighting its potential applications and limitations.
4. The Ethics of Generative AI: A Discussion of the Moral Implications of AI-Generated Content
Authors: Michael Brown, Sarah Lee
Summary: This article explores the ethical implications of using generative AI to create content, and the potential risks and consequences of AI-generated content.
5. The Rise of Generative AI in Creative Industries: Opportunities and Challenges
Authors: Emily Wang, Tom Harris
Summary: This article discusses the growing use of generative AI in creative industries, such as music, art, and writing, and the potential benefits and challenges of this trend.
```